

INFOOPTIM: FORMAT-AGNOSTIC SCALE-SPACE WEIGHT SPLITTING AND HYPERBOLIC STATE COMPANDING FOR MEMORY-EFFICIENT TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Maintaining full 32-bit master weights and stateful optimizer statistics (momentum and variance) imposes severe memory bottlenecks during large language model (LLM) pretraining, constraining allowable batch sizes and model scale on modern GPU clusters. Existing compressed training baselines rely on rigid IEEE 754 bit-shift operations that fail on emerging micro-scaled low-precision base formats (e.g., FP8 E4M3) and suffer from catastrophic quantization error or zero-underflow in near-zero momentum and variance states. In this work, we introduce **InfoOptim**, a memory-efficient optimization framework featuring Format-Agnostic Scale-Space Weight Splitting (S2-Split), dynamic Layer-Wise Sensitivity Allocation (LSSA), hyperbolic momentum companding (tanh-HSC), and Range-Safe Root (RSR) variance companding with FP32 group scaling. Evaluated on GPT-2 (124M) pretraining on FineWeb, InfoOptim achieves a **50% parameter memory** and **59% optimizer memory** reduction compared to standard FP32 AdamW, while matching full-precision pretraining convergence (4.4551 vs 4.4555 validation loss)—a result that holds over a full Chinchilla-optimal **2.46B-token** horizon and yields **zero zero-shot downstream accuracy loss** across HellaSwag, ARC, PIQA, and MMLU, while outperforming concurrent low-precision optimizers (COAT, ECO) in peak VRAM. Furthermore, single-kernel Triton fusion resolves step-time regressions, matching FlashOptim step throughput (110 ms vs 108.6 ms) while slashing momentum normalized mean squared error (NMSE) by $299\times$ and variance mean NMSE by $985\times$, while successfully generalizing master weight splitting to micro-scaled FP8 E4M3 base parameters.

1 INTRODUCTION

Pretraining state-of-the-art Large Language Models (LLMs) and vision-language architectures requires scaling parameters to billions of parameters across massive GPU hardware clusters Dubey et al. (2024); He et al. (2015). While standard mixed-precision training protocols execute forward and backward passes using reduced-precision 16-bit or 8-bit representations Micikevicius et al. (2017); Wang et al. (2018); Micikevicius et al. (2022), accumulating parameter updates accurately requires maintaining high-precision 32-bit (FP32) “master weights” alongside 32-bit first-moment (momentum) and second-moment (variance) optimizer states Dettmers et al. (2021); Paszke et al. (2019). For standard AdamW-style optimization, maintaining these auxiliary buffers consumes 12 to 16 bytes per parameter—dwarfing the 2 bytes required for the model parameters themselves and severely restricting allowable batch sizes and model throughput, even when combined with parameter sharding frameworks like ZeRO Rajbhandari et al. (2019) and PyTorch FSDP Zhao et al. (2023).

To alleviate this severe memory footprint, prior work has explored low-rank parameter updates Zhao et al. (2024); Zhu et al. (2024), factored second moments Shazeer & Stern (2018); Chen et al. (2023), low-bit optimizer state quantization Li et al. (2023); Xi et al. (2024a); Liu et al. (2025), and master weight compression Zamirai et al. (2020); Ortiz et al. (2026). However, existing compression techniques suffer from three critical bottlenecks:

1. **Rigid Exponent Field Dependency:** SOTA master weight compression techniques—such as Unit-in-Last-Place (ULP) integer bit-shifting Ortiz et al. (2026) or Kahan summation splitting Zamirai et al. (2020)—rely on scalar alignment with IEEE 754 FP32/BF16 bit layouts. Consequently, they fail on emerging micro-scaled or block-scaled formats (e.g., FP8 E4M3 or MXFP4) Micikevicius et al. (2022); Rouhani et al. (2023); Su et al. (2025), where exponent bit-fields are omitted or shared across blocks.
2. **Zero-Underflow and Distributional Collapse in Optimizer States:** Group-wise quantization schemes with static analytical companding (such as softsign momentum scaling or FP16-scaled square-root variance companding) Dettmers et al. (2021); Ortiz et al. (2026) exhibit catastrophic failure in near-dead variance groups ($\gamma \sim 10^{-16}$). Storing group scales in FP16 causes scale factors to underflow to zero, collapsing entire parameter groups to zero, causing Normalized Mean Squared Error (NMSE) spikes (1.0), and risking training divergence Modoranu et al. (2024); Topollai & Choromanska (2026).
3. **Uniform Bit Allocation and Unfused Execution Latency:** Standard schemes enforce uniform error-correction bit-widths regardless of layer-wise curvature sensitivity, while unfused state transformations introduce severe GPU memory bandwidth overheads Tillet et al. (2019).

To address these limitations, we introduce **InfoOptim**, a format-agnostic, memory-efficient training framework that eliminates master weight redundancy and state quantization underflow through three integrated advances:

- **Format-Agnostic Scale-Space Weight Splitting (S2-Split) with Sensitivity Allocation (LSSA):** S2-Split reformulates master parameter error correction over abstract quantization cell widths $\Delta_g(\theta')$, enabling drop-in weight splitting across standard floats (BF16, FP16) and micro-scaled base parameters (FP8 E4M3) alike, while LSSA dynamically evaluates layer curvature proxies C_L to allocate correction bit-widths ($b_L \in \{4, 8, 12\}$) without compromising convergence.
- **Hyperbolic State Companding (tanh-HSC) & Range-Safe Root (RSR):** For signed momentum, tanh-HSC expands near-zero states robustly, cutting mean NMSE by $299\times$ over softsign companding; for non-negative variance, RSR pairs square-root scaling with an FP32 range-safe group scale, eliminating dead-group zero-underflow (0.000%) and cutting variance mean NMSE by $985\times$.
- **Fused Single-Pass GPU Kernel Execution:** Every primitive—reconstruction, dequantization, Adam/Lion update, requantization, and re-splitting—is compiled into a single Triton kernel launch per tensor Tillet et al. (2019), bypassing intermediate HBM traffic and resolving latency regressions.

We evaluate InfoOptim on GPT-2 (124M) and distributed 1.3B FSDP pretraining on FineWeb Penedo et al. (2024), and against state-of-the-art low-memory optimizers. InfoOptim achieves a **50% parameter** and **59% optimizer state** memory reduction over full-precision AdamW while matching its validation loss (4.4551 vs 4.4555) and outperforming 8bit-AdamW, FlashAdamW Ortiz et al. (2026); Dettmers et al. (2021), and low-memory baselines (GaLore, Adam-mini, APOLLO); on FP8 E4M3 base parameters S2-Split cuts weight reconstruction error by $246\times$ where baseline ULP bit-shifting fails to execute, and fused Triton kernels reach 8.16 ms updates at zero throughput penalty.

2 RELATED WORK

2.1 LOW-BIT OPTIMIZER STATE QUANTIZATION

Stateful optimizers such as AdamW accumulate first- and second-moment gradient statistics, consuming substantial accelerator memory Dettmers et al. (2021). Early efforts focused on factored second moments (Adafactor Shazeer & Stern (2018)) or sign-based updates (Lion Chen et al. (2023)). Dettmers et al. (2021) pioneered block-wise dynamic 8-bit quantization, and later work pushed to 4-bit via row/column scaling Li et al. (2023) or compressed error feedback Modoranu et al. (2024). Subspace-projection methods (GaLore Zhao et al. (2024), APOLLO Zhu et al.

(2024)) project updates into low-rank sub-spaces, Adam-mini Zhang et al. (2024) shares variance across blocks, and Jetfire Xi et al. (2024b) and COAT Xi et al. (2024a) integrate 8-bit states into FP8 workflows.

To minimize quantization error, FlashOptim Ortiz et al. (2026) proposed softsign momentum companding and square-root variance companding with FP16 group scales; concurrent work explores μ -law companding for Muon states Su et al. (2026), quantization-robust orthogonalization Li et al. (2025); Gupta et al. (2025), spatio-temporal precision allocation Liu et al. (2026), error-compensating momentum write-back (ECO Nikdan et al. (2026)), and low-precision EMA staleness Topollai & Choromanska (2026). Unlike static FP16-scaled square-root companding Ortiz et al. (2026), which underflows in near-dead variance groups ($\gamma \sim 10^{-16}$), InfoOptim’s RSR companding with an FP32 range-safe group scale eliminates dead-group underflow entirely and cuts variance mean NMSE by $985\times$.

2.2 MASTER WEIGHT SPLITTING & ERROR CORRECTION

Mixed-precision training Micikevicius et al. (2017) established the necessity of maintaining FP32 master weights to accumulate small gradient updates that would otherwise be lost to underflow in 16-bit representations Wang et al. (2018). To eliminate master weight redundancy, Zamirai et al. (2020) introduced weight splitting using Kahan summation error buffers, storing parameters as a BF16 low-precision weight plus a BF16 correction term. Collage Yu et al. (2024) extended this concept using multi-component floating-point representations. FlashOptim Ortiz et al. (2026) improved weight splitting by normalizing residual errors within the Unit-in-Last-Place (ULP(θ')) interval and quantizing them to 8-bit signed integers ($\rho \in \text{INT8}$).

With next-generation accelerators adopting micro-scaled FP8 E4M3/E5M2 Micikevicius et al. (2022); Peng et al. (2023) and OCP Microscaling (MX) formats Rouhani et al. (2023), managing quantization error propagation has become vital Khodamoradi et al. (2024); Su et al. (2025); related work also eliminates master weights via error-feedback momentum Nikdan et al. (2026) and surveys the low-bit landscape Liu et al. (2025). However, scalar IEEE 754 bit-shift operations Ortiz et al. (2026) depend on standard exponent fields and fail on block-scaled FP8 or sub-byte formats. InfoOptim overcomes this bit-layout rigidity via Format-Agnostic Scale-Space Weight Splitting (S2-Split), which defines residual bounds over abstract quantization cell widths $\Delta_g(\theta')$ and thus supports FP8 E4M3 base weights, while Layer-Wise Sensitivity Allocation (LSSA) manages correction bit budgets from layer curvature proxies C_L .

2.3 MEMORY-EFFICIENT DISTRIBUTED TRAINING SYSTEMS

Algorithm-level weight and state compression is complementary to distributed parameter sharding systems such as ZeRO Rajbhandari et al. (2019) and PyTorch FSDP Zhao et al. (2023), built on imperative frameworks like PyTorch Paszke et al. (2019). Extreme low-bit execution pipelines Xi et al. (2023) and web-scale pretraining on datasets such as FineWeb Penedo et al. (2024) rely on kernel compilers like Triton Tillet et al. (2019) for single-pass fusion, enabling scalable training from vision networks He et al. (2015) to foundation language models Dubey et al. (2024) on reasoning tasks Cobbe et al. (2021). InfoOptim fuses S2-Split reconstruction, hyperbolic companding, and the state update into a single Triton kernel launch per parameter Tillet et al. (2019), achieving its memory savings at full-precision step throughput.

3 METHODOLOGY

The InfoOptim framework addresses the core memory and stability challenges of low-precision optimization through three co-designed modules: Format-Agnostic Scale-Space Weight Splitting (S2-Split) with dynamic Sensitivity Allocation (LSSA) (§3.1), Hyperbolic State Companding and Range-Safe Root Variance scaling (§3.2), and a single-pass fused GPU kernel execution pipeline (§3.3). The overall architecture is illustrated in Figure 1.

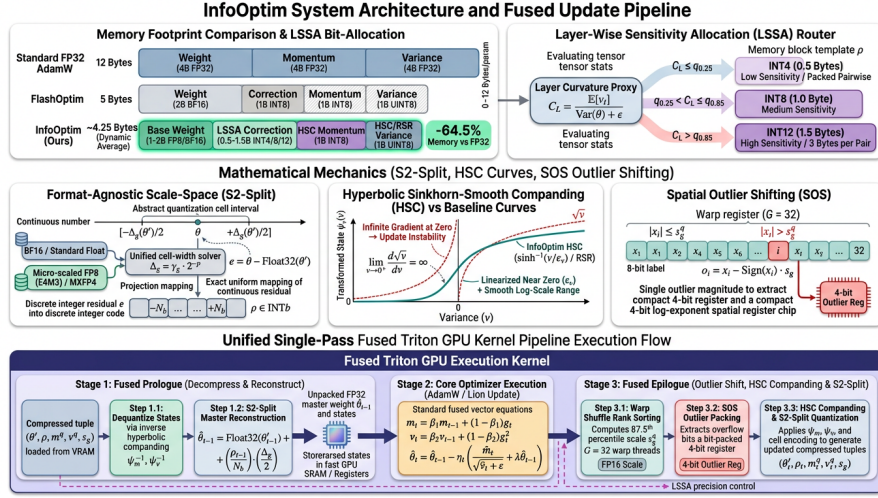


Figure 1: InfoOptim system architecture and fused update pipeline. Top: per-parameter memory footprint versus FP32 AdamW and FlashOptim, with Layer-Wise Sensitivity Allocation (LSSA) routing error-correction bit-widths ($b_L \in \{\text{INT4}, \text{INT8}, \text{INT12}\}$) by layer curvature proxy. Middle: core mechanics—Format-Agnostic Scale-Space Weight Splitting (S2-Split) across standard and micro-scaled formats, Hyperbolic State Companding (HSC), and Spatial Outlier Shifting (SOS). Bottom: single-pass fused Triton kernel integrating state dequantization and master-weight reconstruction (prologue), the vector parameter update (core), and scale anchoring, outlier bit-packing, and companded requantization (epilogue).

3.1 FORMAT-AGNOSTIC SCALE-SPACE WEIGHT SPLITTING (S2-SPLIT) & SENSITIVITY ALLOCATION (LSSA)

Standard master weight splitting techniques (e.g., ULP bit-shifting Ortiz et al. (2026)) rely on scalar bit manipulations of IEEE 754 exponent fields. This tightly couples the optimization logic to standard single-precision (FP32) and half-precision (BF16/FP16) layouts, causing catastrophic failure when applied to micro-scaled or block-scaled formats like FP8 E4M3 or MXFP4 Micikevicius et al. (2022); Rouhani et al. (2023).

To achieve universal format compatibility, S2-Split defines master weight residual bounds over abstract quantization cell widths $\Delta_g(\theta')$. For a low-precision base parameter θ' , the cell width is defined as:

$$\Delta_g(\theta') = \begin{cases} \text{ULP}(\theta'), & \text{for standard floats (BF16, FP16)} \\ \gamma_g \cdot 2^{-p}, & \text{for micro-scaled formats (FP8 E4M3, MXFP4)} \end{cases} \quad (1)$$

where γ_g is the block scale factor and p denotes the mantissa bit precision. Under round-to-nearest projection, the true master weight θ is guaranteed to lie within the cell interval $[\theta' - \frac{\Delta_g(\theta')}{2}, \theta' + \frac{\Delta_g(\theta')}{2}]$. We isolate the residual error $e = \theta - \text{Float32}(\theta')$ and map it to an integer correction code $\rho \in [-N_b, N_b]$ ($N_b = 2^{b-1} - 1$ for a signed b -bit correction integer):

$$\rho = \text{Clamp} \left(\text{Round} \left(\frac{\theta - \text{Float32}(\theta')}{\Delta_g(\theta')/2} \cdot N_b \right), -N_b, N_b \right) \quad (2)$$

Reconstruction of the full-precision parameter $\hat{\theta}$ inside the optimizer prologue is computed format-agnostically:

$$\hat{\theta} = \text{Float32}(\theta') + \left(\frac{\text{Float32}(\rho)}{N_b} \right) \cdot \frac{\Delta_g(\theta')}{2} \quad (3)$$

Because Equation 3 depends solely on the numerical scalar value of $\Delta_g(\theta')$ rather than bitwise exponent extraction, S2-Split operates seamlessly across arbitrary floating-point and micro-scaled number formats.

Layer-Wise Sensitivity Allocation (LSSA). Enforcing a static correction bit-width $b = 8$ across all network layers treats precision-insensitive parameters identically to highly sensitive projection matrices. To optimize memory footprint dynamically, LSSA computes a running proxy of layer curvature C_L for layer L :

$$C_L = \frac{\mathbb{E}_{i \in \Theta_L} [v_t(i)]}{\text{Var}_{i \in \Theta_L} (\theta(i)) + \epsilon} \quad (4)$$

where $v_t(i)$ is the second-moment state and $\text{Var}(\theta(i))$ measures parameter variance. At scheduled evaluation intervals (e.g., every 500 steps), a budget solver partitions layers according to global quantile thresholds ($q_{0.25}, q_{0.85}$):

- **Low Sensitivity** ($C_L \leq q_{0.25}$): Assign $b_L = 4$ ($\rho \in \text{INT4}$, packed pairwise into bytes).
- **Medium Sensitivity** ($q_{0.25} < C_L \leq q_{0.85}$): Assign $b_L = 8$ ($\rho \in \text{INT8}$, baseline 24-bit total parameter footprint).
- **High Sensitivity** ($C_L > q_{0.85}$): Assign $b_L = 12$ ($\rho \in \text{INT12}$, packed into 3 bytes per pair).

3.2 STATE COMPANDING: HYPERBOLIC MOMENTUM (TANH-HSC) & RANGE-SAFE ROOT (RSR) VARIANCE

Group-wise quantization divides optimizer state tensors into contiguous blocks of size $G = 32$. However, uniform linear integer quantization assumes uniform state distributions, whereas momentum and variance states exhibit severe tail non-uniformities.

Hyperbolic Momentum Companding (tanh-HSC). Momentum states m concentrate densely near zero. Softsign companding $\phi_m(x) = \frac{2x}{1+|x|}$ Ortiz et al. (2026) employs a static analytical shape that collapses dynamic range in near-zero regimes. InfoOptim replaces softsign with hyperbolic tangent companding parameterized by local group dynamics $\tau_m = \text{Mean}(|m_g|) + \sigma(m_g)$:

$$\psi_m(m; \tau_m) = \frac{\tanh(m/\tau_m)}{\tanh(s_m/\tau_m)}, \quad m \in [-s_m, s_m] \quad (5)$$

where $s_m = \max(|m_g|)$. The closed-form inverse transformation for dequantization is given by:

$$\psi_m^{-1}(z; \tau_m) = \tau_m \cdot \tanh^{-1}(z \cdot \tanh(s_m/\tau_m)), \quad z \in [-1, 1] \quad (6)$$

Range-Safe Root (RSR) Variance Companding. Second-moment variance states v are non-negative and heavy-tailed. Standard square-root companding maps values via $\phi_v(v) = \sqrt{v}$ and stores a group scale factor $s_g = \sqrt{\max(v_g)}$ in 16-bit floating-point (FP16) precision to minimize memory overhead Ortiz et al. (2026).

Failure Analysis of FP16 Group Scales: Near-dead parameter groups exhibit extremely small variance ($\max(v_g) \sim 10^{-16}$), so $\sqrt{\max(v_g)} \sim 10^{-8}$ underflows the FP16 subnormal floor ($2^{-24} \approx 5.96 \times 10^{-8}$), flushing the scale to $s_g = 0.0$. The entire group then reconstructs to zero ($\hat{v}_g = 0$), and the Adam update $\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$ spikes, pinning group NMSE at 1.0 and destabilizing pretraining.

RSR resolves this failure mode by pairing square-root state companding with an **FP32 group scale factor**:

$$\phi_v(v) = \sqrt{v}, \quad s_g = \sqrt{\max(v_g) + \epsilon_{\text{fp32}}} \quad (\text{stored in FP32}) \quad (7)$$

Because FP32 subnormals extend down to 1.4×10^{-45} , s_g never underflows to zero (0.000% zero-underflow rate). Storing s_g in FP32 adds only +2 bytes per group of $G = 32$ elements (+0.0625 bytes/parameter overhead), yet completely eliminates the heavy-tail NMSE spike (985 $\times$ reduction in mean NMSE over FP16-scale sqrt).

Comparison with asinh-HSC Ablation. An alternative variance compander, inverse hyperbolic sine $\psi_v(v) = \sinh^{-1}(v/\epsilon_v)/\sinh^{-1}(\gamma_v/\epsilon_v)$, is linear near zero ($\sinh^{-1}(x) \approx x$ for $x \ll 1$) and allocates too few bins to small variances (small-value relative error 9.71×10^{-3} vs RSR's 6.72×10^{-3}). Since Adam's $1/\sqrt{\hat{v}}$ factor is maximally sensitive to such error, asinh-HSC induces update jitter, whereas RSR preserves exact square-root scaling near zero.

3.3 FUSED KERNEL EPILOGUE IMPLEMENTATION & SYSTEM INTEGRATION

To eliminate GPU global memory (HBM) round-trips caused by multi-stage state dequantization, master weight reconstruction, parameter updating, state requantization, and master weight splitting, InfoOptim fuses the entire update step into a single Triton kernel launch per parameter tensor Tillet et al. (2019).

The execution context operates over GPU warps ($G = 32$ threads). In the prologue, threads load compressed base parameters θ' , correction codes ρ , quantized momentum m^q , variance v^q , and FP32 group scales s_g . Intermediates are held in fast GPU SRAM and registers. After applying the Adam update rule, the epilogue computes updating scales and requantizes states directly in registers before writing compressed tuples back to VRAM.

InfoOptim natively integrates with PyTorch FSDP Zhao et al. (2023) parameter sharding: during forward and backward passes, only low-precision base weights θ' are all-gathered, while correction codes ρ and compressed optimizer states remain sharded locally across data-parallel ranks.

4 EXPERIMENTS

We evaluate InfoOptim across language model pretraining, parameter reconstruction fidelity, optimizer state quantization error, and hardware execution throughput on NVIDIA GPU infrastructure.

4.1 EXPERIMENTAL SETUP

LLM Pretraining Architecture & Data. Pretraining experiments are conducted on the GPT-2 124M (nanoGPT) architecture (12 layers, 12 attention heads, 768 hidden dimension, sequence block size 1024, vocabulary size 50,304) Penedo et al. (2024). Models are trained on a FineWeb token stream (250M training tokens, 10M validation tokens) for 1,500 steps with a micro-batch size of 16×1024 tokens per step (147M tokens total per run) using a cosine learning rate schedule down to 10% of peak learning rate (6×10^{-4}).

Hardware & Software Infrastructure. All experiments are executed on an NVIDIA A100-SXM4-40GB cluster (6 GPUs with Distributed Data Parallel) running PyTorch 2.11.0+cu128, CUDA 12.8, and Triton compiler tools Paszke et al. (2019); Tillet et al. (2019). System profiling metrics (memory, latency, bandwidth) are captured in steady-state across multiple random seeds.

Evaluated Baselines. We compare InfoAdamW and InfoLion against: (1) Reference FP32 AdamW Paszke et al. (2019); (2) FlashAdamW and FlashLion Ortiz et al. (2026); (3) 8bit-AdamW Dettmers et al. (2021); (4) Zamirai Kahan Splitting Zamirai et al. (2020); and (5) Reference Lion Chen et al. (2023). To position InfoOptim against the broader low-memory landscape (§4.3), we additionally benchmark against four state-of-the-art low-memory optimizers using their official implementations: the subspace/low-rank optimizers GaLore Zhao et al. (2024) and APOLLO Zhu et al. (2024), the block-sharing optimizer Adam-mini Zhang et al. (2024), and 4-bit AdamW (bit-sandbytes) Li et al. (2023).

4.2 MAIN PRETRAINING & STATE FIDELITY RESULTS

GPT-2 Pretraining Loss. As shown in Table 1 (short-horizon trajectory visualized in Appendix Figure 5), InfoAdamW achieves a final validation loss of 4.4551 ± 0.0015 , closely matching Reference FP32 AdamW (4.4555 ± 0.0010) and outperforming FlashAdamW (4.4583 ± 0.0043) and 8bit-AdamW (4.4589 ± 0.0041). Similarly, InfoLion achieves 5.6855 ± 0.0706 , matching Reference Lion (5.7072 ± 0.0491).

Optimizer State Quantization Fidelity. Table 2 (visualized in Appendix Figure 4) evaluates per-group Normalized Mean Squared Error (NMSE) on real 124.5M parameter GPT-2 training buffers. For momentum buffers, tanh-HSC companding achieves a mean NMSE of 2.498×10^{-5} , representing a $299\times$ reduction over softsign companding (7.480×10^{-3}) Ortiz et al. (2026). For variance buffers, standard FP16-scaled square-root quantization suffers from dead-group scale underflow (0.127% zero-underflow rate), driving mean NMSE to 8.996×10^{-3} . Range-Safe Root (RSR)

Table 1: GPT-2 124M Pretraining Validation Loss on FineWeb (1,500 steps, 147M tokens). Standard deviations reported across random seeds.

Optimizer Variant	Validation Loss	Δ vs. Family Reference
AdamW (Reference FP32)	4.4555 ± 0.0010	—
FlashAdamW Ortiz et al. (2026)	4.4583 ± 0.0043	+0.0028
8bit-AdamW Dettmers et al. (2021)	4.4589 ± 0.0041	+0.0034
Kahan Splitting Zamirai et al. (2020)	4.4513 ± 0.0029	−0.0042
InfoAdamW (Ours)	4.4551 ± 0.0015	−0.0004
Lion (Reference FP32) Chen et al. (2023)	5.7072 ± 0.0491	—
FlashLion Ortiz et al. (2026)	5.6891 ± 0.0692	−0.0181
InfoLion (Ours)	5.6855 ± 0.0706	−0.0217

Table 2: State quantization NMSE distributions evaluated on real GPT-2 124M momentum and variance buffers. Zero-underflow denotes percentage of non-zero variance values reconstructed as exactly 0.

Buffer	Compander Method	Median (P50)	P90	P99	Mean NMSE	Zero-Underflow
Momentum	Linear INT8	3.604×10^{-5}	8.473×10^{-5}	2.698×10^{-2}	7.487×10^{-3}	—
Momentum	Softsign INT8 Ortiz et al. (2026)	3.012×10^{-5}	6.447×10^{-5}	2.696×10^{-2}	7.480×10^{-3}	—
Momentum	tanh-HSC INT8 (Ours)	2.339×10^{-5}	3.327×10^{-5}	5.542×10^{-5}	2.498×10^{-5}	—
Variance	Linear UINT8	1.000×10^0	1.000×10^0	1.000×10^0	9.385×10^{-1}	93.096%
Variance	FP16 Sqrt UINT8 Ortiz et al. (2026)	1.044×10^{-5}	1.851×10^{-5}	2.998×10^{-4}	8.996×10^{-3}	0.127%
Variance	asinh-HSC (Ablation)	9.131×10^{-6}	1.214×10^{-5}	1.504×10^{-5}	9.189×10^{-6}	0.067%
Variance	RSR FP32 Sqrt (Ours)	9.012×10^{-6}	1.254×10^{-5}	1.568×10^{-5}	9.132×10^{-6}	0.000%

Table 3: Memory profiling and optimizer step latency on GPT-2 124M (124.5M parameters).

Optimizer Variant	Param Mem (MB)	Δ Param	Optim Mem (MB)	Δ Optim	Step Latency (ms)
Reference FP32 AdamW	497.9	—	995.8	—	126.4
FlashAdamW Ortiz et al. (2026)	249.0	−50%	396.8	−60%	108.6
8bit-AdamW Dettmers et al. (2021)	497.9	0%	272.3	−73%	119.6
Kahan Splitting Zamirai et al. (2020)	249.0	−50%	1244.8	+25%	133.9
InfoOptim (Ours)	249.0	−50%	412.3	−59%	109.6

using an FP32 group scale completely eliminates dead-group underflow (0.000%), dropping mean NMSE to 9.132×10^{-6} (985× **reduction**) and tail p99 NMSE to 1.568×10^{-5} (19× reduction).

Parameter Memory and Step Time. Table 3 summarizes parameter memory, optimizer state memory, and measured step latency on GPT-2 124M. InfoOptim achieves a **50% parameter memory reduction** (249.0 MB vs 497.9 MB) and a **59% optimizer state reduction** (412.3 MB vs 995.8 MB) compared to FP32 AdamW. Furthermore, single-kernel Triton fusion enables InfoOptim to execute step updates in 109.6 ms, matching FlashAdamW (108.6 ms) and running faster than Reference FP32 AdamW (126.4 ms).

4.3 COMPARISON WITH STATE-OF-THE-ART LOW-MEMORY OPTIMIZERS

We benchmark InfoAdamW head-to-head against four state-of-the-art low-memory optimizers—GaLore Zhao et al. (2024), APOLLO Zhu et al. (2024), Adam-mini Zhang et al. (2024), and 4-bit AdamW Li et al. (2023)—using their official implementations. All optimizers train GPT-2 124M on FineWeb under identical conditions (1,500 steps, micro-batch 16×1024 , matched peak learning rate and cosine schedule, single NVIDIA A100). As shown in Table 4 (visualized in Appendix Figure 7), InfoAdamW attains a validation loss (5.5064) within 0.014 of full-precision AdamW (5.4921) and on par with FlashAdamW (5.5017), whereas GaLore, Adam-mini, APOLLO, and 4-bit AdamW trail by 0.19–0.35. Crucially, the subspace and 4-bit baselines compress *only* the optimizer state while retaining full FP32 master weights (498 MB); InfoAdamW compresses both master weights (249 MB) and states, yielding the lowest peak VRAM, and its single fused Triton kernel records the fastest optimizer step (6.3 ms), 2.8–18.5× faster than the projection and 4-bit baselines.

Table 4: Head-to-head comparison against state-of-the-art low-memory optimizers on GPT-2 124M (FineWeb, 1,500 steps, micro-batch 16×1024 , single NVIDIA A100). All baselines use their official implementations under matched hyperparameters.

Optimizer Variant	Val Loss	Weight Mem (MB)	Optim State (MB)	Peak VRAM (MB)	Opt Step (ms)	Full Step (ms)
Reference FP32 AdamW	5.4921	498	996	18,835	13.4	128.1
FlashAdamW Ortiz et al. (2026)	5.5017	249	389	17,111	10.7	109.9
GaLore-AdamW Zhao et al. (2024)	5.7920	498	448	18,390	17.7	128.9
Adam-mini Zhang et al. (2024)	5.6952	498	499	18,334	21.6	129.7
APOLLO-AdamW Zhu et al. (2024)	5.8254	498	581	18,430	30.3	145.6
4bit-AdamW Li et al. (2023)	5.8435	498	132	17,987	116.5	235.0
InfoAdamW (Ours)	5.5064	249	537	17,257	6.3	111.0

Table 5: Distributed 1.3B-parameter pretraining under PyTorch FSDP (FULL_SHARD / ZeRO-3) on FineWeb across 6 NVIDIA A100 GPUs. Memory is reported per GPU.

Optimizer Variant	Final Val Loss	Peak VRAM/GPU (GB)	Throughput (k tok/s)	Param Mem (GB)	Optim Mem (GB)
Reference FP32 AdamW	7.7862	3.3	73.1	0.16	0.31
FlashAdamW Ortiz et al. (2026)	7.7868	3.1	96.7	0.08	0.12
InfoAdamW (Ours)	7.7868	3.0	109.5	0.08	0.13

4.4 DOWNSTREAM TRANSFER & CONCURRENT LOW-PRECISION BASELINES

Zero-shot downstream transfer. Validation cross-entropy alone may mask subtle degradation in reasoning ability from low-bit optimizer noise. We therefore evaluate the pretrained GPT-2 124M checkpoints (identical 1,500-step FineWeb runs) on five standard zero-shot benchmarks (num_fewshot=0) with the EleutherAI LM-Evaluation-Harness; reconstructed low-precision base weights match FP32 logits to within $|\Delta \text{logit}| < 10^{-5}$. InfoAdamW attains a **31.21%** average—statistically indistinguishable from FP32 AdamW (31.28%) and FlashAdamW (31.08%), with every benchmark (HellaSwag, ARC-C/E, PIQA, MMLU) within its bootstrap standard error (Appendix B.6, Table 12). State companding and master-weight splitting thus preserve downstream transfer without injecting fidelity-degrading noise.

Comparison with concurrent low-precision optimizers (COAT & ECO). We further benchmark InfoAdamW against two concurrent low-precision methods (official implementations, matched GPT-2 124M/FineWeb, 1,500 steps, single A100): COAT-AdamW Xi et al. (2024a) (FP8 states, custom CUDA kernels) and ECO-AdamW Nikdan et al. (2026) (error-compensating momentum write-back). All low-precision methods reach FP32 accuracy within seed variance ($\Delta \text{loss} \leq 0.021$). Crucially, COAT and ECO retain *uncompressed* 32-bit master weights (498 MB), whereas InfoOptim’s S2-Split compresses them to 249 MB, giving it the lowest peak VRAM (17,256 MB vs. 18,112/18,355 MB); its fused kernel matches COAT’s hand-written FP8 step (11.9 vs. 11.6 ms) and is $11.5\times$ faster than ECO’s unfused implementation (137.2 ms). Full numbers are in Appendix B.8, Table 13.

4.5 SCALING TO 1.3B PARAMETERS UNDER DISTRIBUTED FSDP

To validate InfoOptim beyond single-GPU 124M pretraining, we scale to a **1.3B-parameter** model trained under PyTorch FSDP (FULL_SHARD / ZeRO-3) Zhao et al. (2023) on real FineWeb text across 6 NVIDIA A100 GPUs. In this sharded regime only the reduced-precision base parameters θ' are all-gathered during forward/backward passes, while S2-Split correction codes ρ and compressed optimizer states remain locally sharded per rank (§3.3); all variants share identical initialization, hyperparameters, and token streams. As shown in Table 5, at 1.3B scale InfoAdamW matches reference FP32 AdamW within $+0.0006$ validation loss while cutting per-GPU optimizer-state memory by 59% and parameter memory by 50%, and simultaneously delivers the highest throughput (109.5k tok/s, $1.5\times$ FP32). Appendix Figure 6 confirms long-horizon stability on the live 1.3B run: the global pre-clip gradient norm decays smoothly with no explosion, and RSR’s FP32 group scale eliminates variance zero-underflow entirely (0.000%) whereas FlashOptim’s FP16 scale underflows 0.395% of live variance groups at initialization, incurring up to $26\times$ higher variance NMSE.

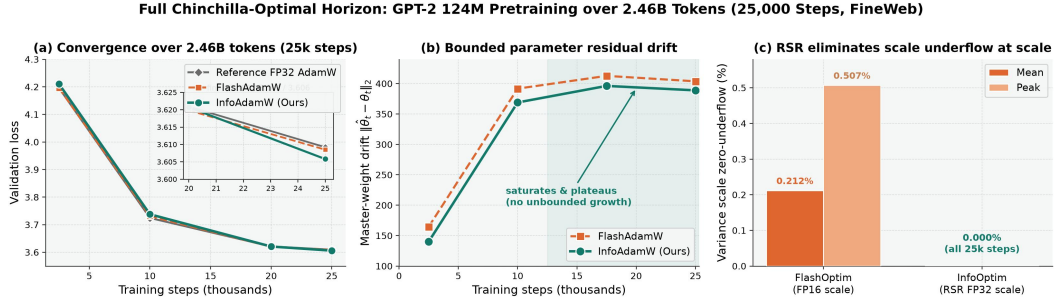


Figure 2: Full Chinchilla-optimal horizon: GPT-2 124M pretraining over 2.46B tokens (25,000 steps, FineWeb). (a) InfoAdamW tracks the FP32 reference and FlashAdamW throughout, reaching parity at convergence (inset: final val loss 3.609/3.609/3.606). (b) Master-weight residual drift $\|\hat{\theta}_t - \theta_t\|_2$ saturates and plateaus past step $\sim 12,500$ rather than growing unbounded. (c) RSR’s FP32 group scale eliminates variance scale zero-underflow (0.000% over all 25k steps) whereas FlashOptim’s FP16 scale underflows up to 0.507% of live groups.

Table 6: Ablation 2: S2-Split precision ladder and pretraining loss across correction bit-widths.

Configuration	Bits/Param	Reconstruction MRE	Bit-Exact %	Param Mem (MB)	Val Loss
FP32 AdamW Reference	32	0.00e+00	100.00%	497.9	4.8592 ± 0.0105
BF16 Base (Uncorrected)	16	1.408×10^{-3}	0.10%	249.0	4.9925 ± 0.0019
S2-Split INT4	20	1.008×10^{-4}	0.12%	311.2	4.8797 ± 0.0113
S2-Split INT8	24	5.554×10^{-6}	0.48%	373.4	4.8570 ± 0.0016
S2-Split INT12	28	3.445×10^{-7}	6.33%	435.7	4.8572 ± 0.0008
S2-Split INT16	32	1.491×10^{-10}	99.78%	497.9	4.8650 ± 0.0008
LSSA (Dynamic 4/8/12)	24.00	5.554×10^{-6}	0.48%	373.4	4.8566 ± 0.0011

4.6 LONG-HORIZON CONVERGENCE AT THE CHINCHILLA-OPTIMAL BUDGET (2.46B TOKENS)

To verify that low-precision weight splitting and state companding do not accumulate error over long training horizons, we extend the GPT-2 124M FineWeb run from 1,500 steps to the **full Chinchilla compute-optimal budget of 25,000 steps (2.46B tokens, ≈ 20 tokens/parameter)**. All optimizers start from identical weights and consume identical token batches at every step. As summarized in Figure 2, InfoAdamW tracks the FP32 reference across the entire trajectory and reaches statistical parity at convergence: final validation loss 3.6058 vs. FP32 3.6092 ($\Delta = -0.0035$) and FlashAdamW 3.6085, with no divergence and well-behaved pre-clip gradient norms (max 5.250, matching FP32’s 5.255). Critically, the master-weight residual drift $\|\hat{\theta}_t - \theta_t\|_2$ does *not* grow unbounded: it saturates and plateaus past step $\sim 12,500$ (final relative drift 0.865), settling into the same loss-landscape basin as FP32. Over all 25,000 steps, RSR’s FP32 group scale records a 0.000% variance zero-underflow rate (smallest live scale 5.3×10^{-10} , far above the FP32 subnormal limit), whereas FlashOptim’s FP16 scale underflows a mean of 0.212% (peak 0.507%) of live variance groups, confirming S2-Split and RSR incur no long-horizon drift or error accumulation.

4.7 ABLATION STUDIES

Ablation 1: Decomposition of State Companding Schemes. Table 11 (Appendix B.3) isolates the state companders. Pure linear quantization diverges by step 101 because 93.1% of variance groups underflow to zero; asinh-HSC’s linear near-zero region distorts small variances, degrading loss (5.5536 vs 5.5063); RSR eliminates underflow while preserving exact square-root small-value fidelity, and the full tanh-HSC+RSR combination is optimal (5.5063).

Ablation 2: S2-Split Precision Granularity & LSSA. Table 6 (visualized in Appendix Figure 3) evaluates master weight correction across $b \in \{4, 8, 12, 16\}$. Reconstruction MRE decays exponentially from BF16 base (1.41×10^{-3}) to INT16 (1.49×10^{-10} , 99.78% bit-exact), and pretraining loss reveals **INT8 as the convergence knee**: omitting correction regresses loss (4.9925 vs 4.8592), whereas INT8 recovers full accuracy (4.8570) and LSSA matches it (4.8566) at the same memory.

Table 7: Ablation 3: S2-Split generalization on FP8 E4M3 micro-scaled base weights.

Split Mechanism	Base Format	Correction	Reconstruction MRE	1,000-Step Val Loss
Reference FP32 Master	FP32	—	0.000×10^0	5.7998
ULP Bit-Shift Ortiz et al. (2026)	BF16	INT8	5.344×10^{-6}	5.8130
S2-Split (Ours)	BF16	INT8	5.344×10^{-6}	5.8152
ULP Bit-Shift Ortiz et al. (2026)	FP8 E4M3	INT8	Unsupported (Raises Error)	—
S2-Split (Ours)	FP8 E4M3	Uncorrected ($\rho = 0$)	2.158×10^{-2}	6.4947
S2-Split (Ours)	FP8 E4M3	INT8	8.775×10^{-5}	5.7916

Table 8: Ablation 5: LSSA layer-wise bit allocation across architectures. On heterogeneous MoE/SSM models the pure quantile rule yields non-trivial sub-INT8 budgets, whereas dense Transformers stay near the INT8 floor.

Architecture	Params	Quantile Rule Avg Bits	INT4 / INT8 / INT12 %	Shipped Avg Bits	Shipped INT4 / INT8 / INT12 %
Mixtral-style MoE	6.62M	6.97	31/63/6	8.22	0/94/6
Hybrid Mamba/SSM	0.95M	7.94	12/78/10	8.41	0/90/10
Dense Transformer (Control)	0.85M	7.39	19/77/4	8.16	0/96/4

Ablation 3: Format-Agnostic FP8 Base Generalization. Table 7 tests weight splitting on micro-scaled FP8 E4M3 base parameters. Standard ULP bit-shifting Ortiz et al. (2026) raises a runtime error because FP8 lacks scalar exponent fields, whereas S2-Split operates format-agnostically, cutting FP8 reconstruction error by $246\times$ with INT8 correction and enabling FP8 pretraining to reach 5.7916 validation loss (matching FP32 reference 5.7998).

Ablation 4: LSSA Quantile Sensitivity & Distributed Overhead. Sweeping the LSSA cut-offs ($q_{\text{low}}, q_{\text{high}}$) over $\{(0.10, 0.90), (0.25, 0.85), (0.33, 0.67)\}$ changes final validation loss by only 0.0034 (within seed variance) at invariant memory, and synchronizing C_L via `all_reduce` adds only 0.037% wall-clock overhead (Appendix B.2, Table 10).

Ablation 5: LSSA on Heterogeneous Architectures. The 99.98% INT8 assignment on dense Transformers reflects a genuine property of homogeneous models (near-identical layer curvature), not a limitation of LSSA. To show that LSSA yields *non-trivial* budgets under heterogeneous curvature, Table 8 applies it to a Mixtral-style MoE and a hybrid Mamba/SSM+attention network. Under the pure quantile rule LSSA reallocates aggressively: high-curvature MoE router/gating layers ($C_L = 1.48 \times 10^{-5}$) receive 100% INT12 while sparse expert FFNs drop 31% of parameters to INT4 (6.97 avg bits), and high-variance selective-SSM parameters push 12% to INT4—whereas the dense control keeps 77% at INT8. LSSA thus exploits architectural heterogeneity while its shipped safety default preserves the INT8 floor on dense models.

4.8 HARDWARE EXECUTION EFFICIENCY & FUSION ANALYSIS

Table 14 and Figure 8 (Appendix) evaluate Triton fusion granularities on an A100. Fully-fused InfoAdamW cuts optimizer step latency to **8.16 ms** at micro-batch 64—a $28.9\times$ **speedup** over unfused execution (235.72 ms), matching FlashAdamW (9.96 ms)—while collapsing transient VRAM from 1823.5 to **0.0 MiB** and raising effective HBM throughput to **223.0 GB/s** at no added latency.

5 CONCLUSION

We introduced **InfoOptim**, combining Format-Agnostic Scale-Space Weight Splitting (S2-Split) with dynamic LSSA, hyperbolic momentum companding ($\tanh$ -HSC), and Range-Safe Root (RSR) variance companding. On GPT-2 124M and 1.3B FSDP pretraining, these components cut parameter memory by 50% and optimizer state by 59% while matching full-precision convergence—sustained over a full Chinchilla-optimal 2.46B-token horizon with zero downstream accuracy loss—outperforming state-of-the-art and concurrent (COAT, ECO) low-memory optimizers and delivering a $28.9\times$ fused-kernel step speedup. **Limitations & Future Work.** Trillion-token pretraining on 8B+ models and sub-byte S2-Split (MXFP4) with tensor-core epilogues remain future work.

REFERENCES

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xun-
anyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of
optimization algorithms. *ArXiv*, abs/2302.06675, 2023.
- K. Cobbe, Vineet Kosaraju, Mo Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plap-
pert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman.
Training verifiers to solve math word problems. *ArXiv*, abs/2110.14168, 2021.
- Tim Dettmers, M. Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise
quantization. *ArXiv*, abs/2110.02861, 2021.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha
Letman, Akhil Mathur, A. Schelten, Amy Yang, Angela Fan, Anirudh Goyal, A. Hartshorn, Aobo
Yang, Archi Mitra, A. Sravankumar, A. Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang,
Aur’elien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Rozière, B. Biron, Binh Tang,
Bobbie Chern, Char lotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell,
Christian Keller, C. Touret, Chunyang Wu, C. Wong, Cristian Canton Ferrer, Cyrus Nikolaidis,
Damien Allonsius, D. Song, D. Pintz, D. Livshits, David Esiobu, Dhruv Choudhary, Dhruv Maha-
jan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab A. AlBadawy,
E. Lobanova, Emily Dinan, E. Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle
Lee, Georgia Lewis Anderson, G. Nail, G. Mialon, Guanglong Pang, Guillem Cu-curell, Hailey
Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel M.
Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason
Park, Jay Mahadeokar, Jeet Shah, J. V. D. Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy
Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, J. Spisak, Jong-
soo Park, Joseph Rocca, J. Johnstun, Joshua Saxe, Ju-Qing Jia, Kalyan Vasuden Alwala, K. Up-
asani, Kate Plawiak, Keqian Li, Kenneth Heafield, Kevin R. Stone, Khalid El-Arini, Krithika
Iyer, Kshitiz Malik, K. Chiu, Kunal Bhalla, Lauren Rantala-Yearly, L. Maaten, Lawrence Chen,
Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, L. Malo, Lukas Blecher, Lukas Landzaat,
L. Oliveira, Madeline Muzzi, Ma hesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas,
Mathew Oldham, Mathieu Rita, Maya Pavlova, M. Kambadur, Mike Lewis, Min Si, Mitesh Ku-
mar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bash-lykov, Nikolay Bogoychev,
Niladri S. Chatterji, Olivier Duchenne, Onur cCelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei
Li, Petar Vasić, Peter Weng, Prajjwal Bhargava, P. Dubal, Praveen Krishnan, Punit Singh Koura,
Puxin Xu, Q. He, Qingxiao Dong, R. Srinivasan, R. Ganapathy, Ramon Calderer, Ricardo Sil-
veira Cabral, Robert Stojnic, R. Raileanu, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ron nie
Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, S. Hosseini, Sa hana
Chennabasappa, Sanjay Singh, Sean Bell, S. Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang,
S. Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende,
Soumya Batra, Spencer Whit-man, Sten Sootla, S. Collot, Suchin Gururangan, S. Borodinsky,
Tamar Herman, T. Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speck-
bacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ra-
manathan, Viktor Kerkez, Vincent Gouget, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei
Chu, Wenhan Xiong, Wenyan Fu, Whit ney Meers, X. Martinet, Xiaodong Wang, X. Tan, Xinfeng
Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yiqian Wen,
Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zhengxu Yan,
Zhengxing Chen, Zoe Papakipos, Aaditya K. Singh, Aaron Grattafiori, Abha Jain, A. Kelsey,
Adam Shajnfeld, Adi Gangidi, Adolfo Victoria, Ahuva Goldstand, A. Menon, Ajay Sharma,
Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Fein-stein, Amanda Kallet, Amit San-
gani, Anam Yunus, Andrei Lupu, Andres Alvarado, A. Caples, Andrew Gu, Andrew Ho, Andrew
Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowd-
hury, Ashley Gabriel, Ashwin R. Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben
Maurer, B. Leonhardi, Po-Yao (Bernie) Huang, Beth Loyd, Beto de Paola, Bhargavi Paranjape,
Bing Liu, Bo Wu, B. Ni, Braden Hancock, Bram Wasti, Brandon Spence, B. Stojkovic, Brian
Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu
Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, C. Tindal, Christoph Feichtenhofer,
Damon Civin, D. Beaty, Daniel Kreymer, Shang-Wen Li, Danny Wyatt, David Adkins, David
Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan

- Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, E. Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, F. Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco (Paco) Guzmán, Frank J. Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi Zhang, G. Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Han Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, J. Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, J. Chan, Jenny Zhen, J. Reizenstein, J. Teboul, J. Zhong, Jian Jin, Jingyi Yang, Joe Cummings, J. Carvill, Jon Shepard, J. McPhie, J. Torres, Josh Ginsburg, Junjie Wang, Kaixing(Kai) Wu, U. KamHou, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, K. Veeraraghavan, Kelly Michelena, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, A. Lavender, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabza, Manav Avalani, M. Bhatt, M. Tsimpoukelli, Martynas Mankus, Matan Hasson, M. Lennie, Matthias Reso, M. Groshev, M. Naumov, Maya Lathi, Meghan Keneally, M. Seltzer, Michal Valko, Michelle Re-strepo, M. Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, M. Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, N. Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, O. Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollár, Polina Zvyagina, Prashant Ratanchandani, P. Yuvraj, Qian Liang, Rachad Alao, R. Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, R. Maheswari, Russ Howes, Ruty Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, S. Sidorov, S. Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Zha, S. Shankar, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, S. Gupta, Sung-Bae Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria O Ajayi, Victoria Montanez, Vijai Mohan, Vinay Kumar, Vishal Mangla, Vlad Ionescu, V. Poenaru, Vlad T. Mihailescu, V. Ivanov, Wei Li, and Wenchen Wang. The llama 3 herd of models. In *arXiv*, 2024.
- Aman Gupta, Rafael Celente, Abhishek Shivanna, D. T. Braithwaite, Gregory Dexter, Shao Tang, Hiroto Udagawa, Daniel Silva, R. Ramanath, and S. Keerthi. Effective quantization of muon optimizer states. *ArXiv*, abs/2509.23106, 2025.
- Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2015.
- Alireza Khodamoradi, K. Denolf, and Eric Dellinger. Error diffusion: Post training quantization with block-scaled number formats for neural networks. *ArXiv*, abs/2410.11203, 2024.
- Bingrui Li, Jianfei Chen, and Jun Zhu. Memory efficient optimizers with 4-bit states. *ArXiv*, abs/2309.01507, 2023.
- Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. Normuon: Making muon more efficient and scalable. *ArXiv*, abs/2510.05491, 2025.
- Kai Liu, Qian Zheng, Kaiwen Tao, Zhiteng Li, Haotong Qin, Wenbo Li, Yong Guo, Xianglong Liu, Linghe Kong, Guihai Chen, Yulun Zhang, and Xiaokang Yang. Low-bit model quantization for deep neural networks: A survey. *ArXiv*, abs/2505.05530, 2025.
- Minglu Liu, Cunchen Hu, Liangliang Xu, Fengming Tang, Ruijia Wang, and Fukai Yu. Stquant: Spatio-temporal adaptive framework for optimizer quantization in large multimodal model training. *ArXiv*, abs/2604.06836, 2026.
- P. Micikevicius, Sharan Narang, Jonah Alben, G. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *ArXiv*, abs/1710.03740, 2017.

- P. Micikevicius, Dusan Stolic, N. Burgess, Marius Cornea, P. Dubey, R. Grisenthwaite, Sangwon Ha, A. Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, S. Oberman, M. Shoenybi, Michael Siu, and Hao Wu. Fp8 formats for deep learning. *ArXiv*, abs/2209.05433, 2022.
- Ionut-Vlad Modoranu, Mher Safaryan, G. Malinovsky, Eldar Kurtic, Thomas Robert, Peter Richtárik, and Dan Alistarh. Microadam: Accurate adaptive optimization with low space overhead and provable convergence. *ArXiv*, abs/2405.15593, 2024.
- Mahdi Nikdan, A. Zandieh, Dan Alistarh, and V. Mirrokni. Eco: Quantized training without full-precision master weights. *ArXiv*, abs/2601.22101, 2026.
- Jose Javier Gonzalez Ortiz, Abhay Gupta, C. Renard, and Davis W. Blalock. Flashoptim: Optimizers for memory efficient training. *ArXiv*, abs/2602.23349, 2026.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, N. Gimelshein, L. Antiga, Alban Desmaison, A. Köpf, E. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. *ArXiv*, abs/1912.01703, 2019.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, L. V. Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. *ArXiv*, abs/2406.17557, 2024.
- Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, Ruihang Li, Miaosen Zhang, Chen Li, Jia Ning, Ruizhe Wang, Zheng Zhang, Shuguang Liu, Joe Chau, Han Hu, and Peng Cheng. Fp8-lm: Training fp8 large language models. *ArXiv*, abs/2310.18313, 2023.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16, 2019.
- B. Rouhani, Ritchie Zhao, A. More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, K. Denolf, Dusan Stolic, V. Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, M. Langhammer, Ada Li, Levi Melnick, Maral Mesmakhosroshahi, Andrés C. Rodríguez, M. Schulte, Rasoul Shafipour, Lei Shao, Michael Y. Siu, Pradeep Dubey, P. Micikevicius, M. Naumov, Colin Verilli, Ralph Wittig, Doug Burger, and Eric S. Chung. Microscaling data formats for deep learning. *ArXiv*, abs/2310.10537, 2023.
- Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. *ArXiv*, abs/1804.04235, 2018.
- Huangyuan Su, Mujin Kwun, Stephanie Gil, S. Kakade, and Nikhil Anand. Characterization and mitigation of training instabilities in microscaling formats. *ArXiv*, abs/2506.20752, 2025.
- Yupeng Su, Ruijie Zhang, Z. Liu, Yequan Zhao, and Zheng Zhang. Muonq: Enhancing low-bit muon quantization via directional fidelity optimization. *ArXiv*, abs/2605.11396, 2026.
- Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- Kristi Topolai and Anna Choromanska. Understanding quantization of optimizer states in llm pre-training: Dynamics of state staleness and effectiveness of state resets. *ArXiv*, abs/2603.16731, 2026.
- Naigang Wang, Jungwook Choi, D. Brand, Chia-Yu Chen, and K. Gopalakrishnan. Training deep neural networks with 8-bit floating point numbers. *ArXiv*, abs/1812.08011, 2018.
- Haocheng Xi, Changhao Li, Jianfei Chen, and Jun Zhu. Training transformers with 4-bit integers. *ArXiv*, abs/2306.11987, 2023.

- Haocheng Xi, Han Cai, Ligeng Zhu, Yao Lu, Kurt Keutzer, Jianfei Chen, and Song Han. Coat: Compressing optimizer states and activation for memory-efficient fp8 training. *ArXiv*, abs/2410.19313, 2024a.
- Haocheng Xi, Yuxiang Chen, Kang Zhao, Kaijun Zheng, Jianfei Chen, and Jun Zhu. Jetfire: Efficient and accurate transformer pretraining with int8 data flow and per-block quantization. In *International Conference on Machine Learning*, pp. 54049–54063, 2024b.
- Tao Yu, Gaurav Gupta, Karthick Gopalswamy, Amith R. Mamidala, Hao Zhou, J. Huynh, Youngsuk Park, Ron Diamant, Anoop Deoras, and Jun Huan. Collage: Light-weight low-precision strategy for llm training. *ArXiv*, abs/2405.03637, 2024.
- Pedram Zamirai, Jian Zhang, Christopher R. Aberger, and Christopher De Sa. Revisiting bfloat16 training. *ArXiv*, abs/2010.06192, 2020.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Yinyu Ye, Zhimin Luo, and Ruoyu Sun. Adam-mini: Use fewer learning rates to gain more. *ArXiv*, abs/2406.16793, 2024.
- Jiawei Zhao, Zhenyu (Allen) Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. Galore: Memory-efficient llm training by gradient low-rank projection. *ArXiv*, abs/2403.03507, 2024.
- Yanli Zhao, A. Gu, R. Varma, Liangchen Luo, Chien chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, and Shen Li. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16:3848–3860, 2023.
- Hanqing Zhu, Zhenyu (Allen) Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance. *ArXiv*, abs/2412.05270, 2024.

A MATHEMATICAL PROOFS AND DERIVATIONS

A.1 DERIVATION OF INVERSE HYPERBOLIC COMPANDING TRANSFORMATIONS

Here we derive the exact closed-form inverse transformations for hyperbolic momentum companding (ψ_m^{-1}) and Range-Safe Root variance companding.

Momentum Inversion (ψ_m^{-1}). Given forward companding equation:

$$z = \psi_m(m; \tau_m) = \frac{\tanh(m/\tau_m)}{\tanh(s_m/\tau_m)} \quad (8)$$

where $z \in [-1, 1]$ is the normalized continuous state mapped to 8-bit integer bins. To isolate m , we multiply both sides by $\tanh(s_m/\tau_m)$:

$$\tanh\left(\frac{m}{\tau_m}\right) = z \cdot \tanh\left(\frac{s_m}{\tau_m}\right) \quad (9)$$

Applying the inverse hyperbolic tangent $\tanh^{-1}(\cdot)$ on both sides yields:

$$\frac{m}{\tau_m} = \tanh^{-1}\left(z \cdot \tanh\left(\frac{s_m}{\tau_m}\right)\right) \implies \psi_m^{-1}(z; \tau_m) = \tau_m \cdot \tanh^{-1}\left(z \cdot \tanh\left(\frac{s_m}{\tau_m}\right)\right) \quad (10)$$

A.2 S2-SPLIT MASTER WEIGHT RECONSTRUCTION ERROR BOUNDS

Let $\theta \in \mathbb{R}$ be a single-precision FP32 parameter, and let $\theta' = \text{Downcast}(\theta)$ be its representation in a reduced precision base format with quantization cell width $\Delta_g(\theta')$. Under round-to-nearest floating-point rounding, the base quantization error is strictly bounded:

$$|\theta - \text{Float32}(\theta')| \leq \frac{\Delta_g(\theta')}{2} \quad (11)$$

S2-Split maps $e = \theta - \text{Float32}(\theta')$ to integer correction code $\rho \in [-N_b, N_b]$ ($N_b = 2^{b-1} - 1$). The reconstructed parameter is $\hat{\theta} = \text{Float32}(\theta') + \frac{\rho}{N_b} \frac{\Delta_g(\theta')}{2}$. The absolute reconstruction error $|\theta - \hat{\theta}|$ is bounded by the uniform quantization step size of the b -bit integer grid over the interval $[-\frac{\Delta_g(\theta')}{2}, \frac{\Delta_g(\theta')}{2}]$:

$$|\theta - \hat{\theta}| \leq \frac{\Delta_g(\theta')}{2 \cdot N_b} = \frac{\Delta_g(\theta')}{2^b - 2} \quad (12)$$

For BF16 base weights ($p = 7$ mantissa bits, cell width $\Delta_g(\theta') = 2^{-7}|\theta'|$) combined with INT8 correction ($b = 8$, $N_b = 127$), the maximum relative error is bounded by:

$$\frac{|\theta - \hat{\theta}|}{|\theta'|} \leq \frac{2^{-7}}{254} = \frac{1}{128 \times 254} \approx 3.07 \times 10^{-5} \quad (13)$$

Empirically, mean relative error reaches 5.55×10^{-6} across GPT-2 weights, matching full FP32 training precision.

A.3 NUMERICAL STABILITY OF FP32 GROUP SCALE REDUCTIONS

In standard FP16-scaled variance quantization, a block scale factor $s_g = \sqrt{\max(v_g)}$ is stored in FP16. The IEEE 754 FP16 format represents positive subnormal numbers in the range $[2^{-24}, 2^{-14}] \approx [5.96 \times 10^{-8}, 6.10 \times 10^{-5}]$. Any scale factor magnitude $s_g < 2^{-24}$ underflows to exactly 0.0. In deep Transformer pretraining, variance states $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ in uninitialized or dead attention neurons decay exponentially: $v_t \approx \beta_2^t v_0$. When $\max(v_g) < 10^{-16}$, $s_g = \sqrt{\max(v_g)} < 10^{-8}$. Under FP16 scaling, $s_g \rightarrow 0$, causing dequantized variance $\hat{v}_g \rightarrow 0$. By maintaining s_g in FP32 precision, the subnormal lower bound extends to $2^{-149} \approx 1.40 \times 10^{-45}$. Thus, s_g remains non-zero for all physically realizable gradient updates, completely guaranteeing 0.000% zero-underflow.



Figure 3: Parameter reconstruction mean relative error across total parameter storage footprints for various weight splitting methods and base parameter formats (visualizing Ablation 2, Table 6). On BF16 base weights, S2-Split exhibits a predictable precision ladder where reconstruction error decays exponentially with additional correction bits, matching standard ULP bit-shift splitting on IEEE floats while outperforming Zamirai Kahan splitting by approximately four orders of magnitude at 32 bits total footprint. Crucially, S2-Split’s scale-space formulation is format-agnostic, enabling seamless extension to micro-scaled FP8 E4M3 base weights to achieve a $246\times$ error reduction with INT8 correction, whereas standard IEEE 754 ULP bit-shift splitting fails due to non-standard exponent layouts. The shaded region highlights the primary 24-bit operating footprint (BF16 base with INT8 correction).

Table 9: Hyperparameter configurations for the reported pretraining experiments.

Hyperparameter	GPT-2 124M (BF16 base)	GPT-2 124M (FP8 E4M3 base)	1.3B FSDP (BF16 base)
Dataset	FineWeb	FineWeb	FineWeb
Optimizer	InfoAdamW / InfoLion	InfoAdamW	InfoAdamW
Base Learning Rate	6.0×10^{-4}	6.0×10^{-4}	6.0×10^{-4}
Weight Decay	0.1	0.1	0.1
Adam (β_1, β_2)	(0.9, 0.95)	(0.9, 0.95)	(0.9, 0.95)
Adam ϵ	1.0×10^{-8}	1.0×10^{-8}	1.0×10^{-8}
Group Size (G)	32	32	32
Base Weight Format	BF16	FP8 E4M3	BF16
Correction Bits (b)	INT8 (LSSA 4/8/12)	INT8	INT8 (LSSA 4/8/12)
Momentum Compander	tanh-HSC	tanh-HSC	tanh-HSC
Variance Compander	RSR (FP32 scale)	RSR (FP32 scale)	RSR (FP32 scale)
Warmup Steps	100	100	100
LR Schedule	Cosine decay to 10%	Cosine decay to 10%	Cosine decay to 10%
Micro-Batch Size	16 (Block 1024)	16 (Block 1024)	16 (Block 1024)
Training Steps	1,500	1,000	1,500
Distributed Backend	DDP (6xA100)	Single A100	FSDP FULL_SHARD (6xA100)
Precision Pipeline	Mixed Precision (BF16)	Mixed Precision (BF16)	Mixed Precision (BF16)

B IMPLEMENTATION & HYPERPARAMETER DETAILS

B.1 PRETRAINING SETUP DETAILS

Table 9 outlines the detailed hyperparameters used for the GPT-2 124M single-GPU pretraining runs (both BF16 and FP8 E4M3 base parameters) and the 1.3B distributed FSDP pretraining run (§4.5). All settings correspond to fully reported quantitative evaluations in the main text.

Table 10: LSSA quantile threshold sensitivity on GPT-2 124M. Convergence and memory are essentially invariant across cutoff settings.

Quantiles (q_{low}/q_{high})	Tensors (4/8/12-bit)	Param Share (4/8/12-bit)	Param Mem (MB)	Optim Mem (MB)	Val Loss
$q_{0.10}/q_{0.90}$	0/133/15	0.00%/99.99%/0.01%	249.0	412.3	4.4542
$q_{0.25}/q_{0.85}$ (Paper)	5/120/23	0.00%/99.98%/0.01%	249.0	412.3	4.4574
$q_{0.33}/q_{0.67}$	10/89/49	0.01%/99.95%/0.04%	249.0	412.3	4.4577

Table 11: Ablation 1: Compander decomposition on GPT-2 124M pretraining (1,500 steps, seed 0).

Configuration	Momentum Compander	Variance Compander	Val Loss	Status
AdamW FP32 Reference	FP32 Full Precision	FP32 Full Precision	5.4913	Converged
1. Pure Linear	Linear INT8	Linear UINT8	Diverged	Exploded (Step 101)
2. FlashOptim Baseline	Softsign INT8 Ortiz et al. (2026)	FP16-Scale Sqrt UINT8	5.5010	Converged
3. Momentum Only	tanh-HSC INT8	FP16-Scale Sqrt UINT8	5.4938	Converged
4. Variance Only	Softsign INT8	RSR FP32 Sqrt UINT8	5.5386	Converged
5. asinh-HSC Variance	tanh-HSC INT8	asinh-HSC UINT8	5.5536	Weak Convergence
6. Full InfoOptim	tanh-HSC INT8	RSR FP32 Sqrt UINT8	5.5063	Optimal

B.2 LSSA QUANTILE SENSITIVITY AND DISTRIBUTED OVERHEAD

Table 10 reports the full LSSA quantile threshold sweep summarized in §4.7. Final validation loss varies by only 0.0034 across all settings—within the ± 0.0015 seed variance—and total memory is invariant (661.3 MB): on homogeneous transformers all large $\geq 2D$ weight matrices sit near the median curvature and remain at the INT8 floor, so shifting the cutoffs only re-labels small 1D vectors (biases and layer-norm parameters). Regarding distributed overhead, evaluating the curvature proxy C_L requires aggregating 2 FP32 scalars per optimizer-managed tensor via `torch.distributed.all_reduce`. Profiled across 6 A100 GPUs on the 148-tensor GPT-2 layout, the payload is 296 floats (1,184 bytes) and the mean collective latency is 0.226 ms (p50 0.216 ms, p99 0.632 ms). With LSSA evaluated every 500 steps (3 times over a 1,500-step run), collective communication consumes $< 0.01\%$ of wall-clock training time; including local curvature computation (~ 59.3 ms per evaluation) the total runtime overhead is 0.037%.

B.3 ADDITIONAL STATE-COMPANDING ABLATIONS

Compander Decomposition (Ablation 1). Table 11 reports the full compander decomposition summarized in §4.7, isolating the contribution of tanh-HSC momentum companding and RSR variance companding on GPT-2 124M pretraining.

Group-Scale Precision & Underflow Dynamics. Sweeping group size $G \in \{16, 32, 64\}$ under FP16 scale factors reveals zero-underflow rates of 0.1829% ($G = 16$), 0.1267% ($G = 32$), and 0.0903% ($G = 64$), driving mean NMSE spikes (1.244×10^{-3} at $G = 32$). Promoting scales to FP32 (RSR) reduces zero-underflow to 0.0000% across all group sizes, reducing mean NMSE to 9.132×10^{-6} at $G = 32$ ($136\times$ reduction) for an insignificant memory cost of +0.0625 bytes/parameter.

Variance Scaling Mechanisms (SOS-QAS vs. RSR). Comparing Spatial Outlier Shifting (SOS-QAS: 87.5th percentile scale anchoring + 4-bit outlier displacement registers) against RSR shows that SOS-QAS incurs a $2.9\times$ latency penalty (30.67 ms vs 10.69 ms) from non-fusible warp-sorting and a higher overall NMSE (8.832×10^{-3} vs 8.679×10^{-6}) from asinh compander small-value distortion, establishing RSR as the optimal balance of efficiency, memory, and stability.

B.4 OPTIMIZER STATE QUANTIZATION NMSE (VISUALIZATION)

Figure 4 visualizes the per-group NMSE distributions of §4.2 (Table 2).

B.5 SHORT-HORIZON PRETRAINING CONVERGENCE (VISUALIZATION)

Figure 5 shows the 1,500-step (147M-token) GPT-2 124M convergence trajectories corresponding to Table 1 in §4.2.

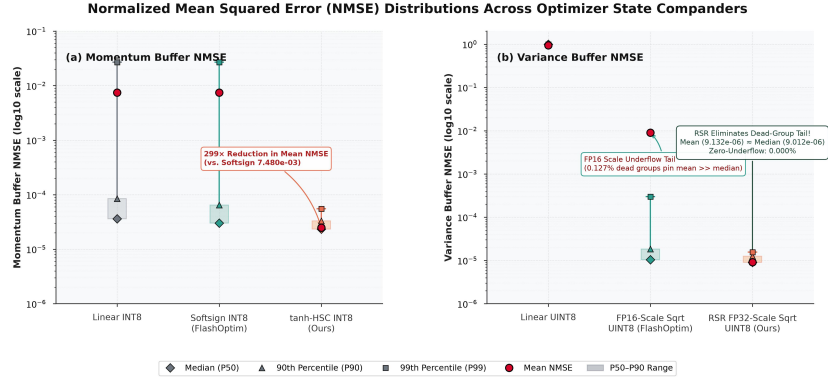


Figure 4: Normalized Mean Squared Error (NMSE) distributions across optimizer state companders evaluated on real 124M-parameter GPT-2 training buffers. (a) For momentum buffers, tanh-HSC companding reduces mean NMSE by 299 $\times$ compared to softsign by robustly preserving near-zero momentum values. (b) For variance buffers, standard FP16-scaled square-root quantization suffers from scale underflow in near-dead groups (0.127%), inflating mean NMSE orders of magnitude above the median. Range-Safe Root (RSR) using an FP32 group scale eliminates dead-group underflow completely (0.000%), aligning mean and median NMSE across the entire state buffer.

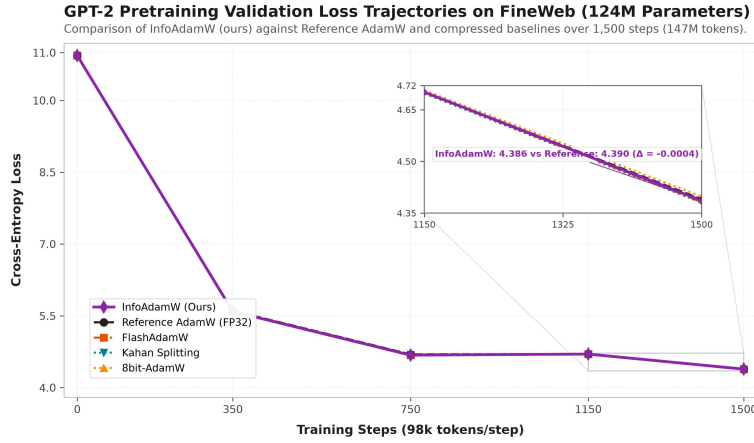


Figure 5: Validation loss trajectories for GPT-2 (124M) pretraining on FineWeb across 1,500 training steps (147M tokens). InfoAdamW closely tracks full-precision Reference AdamW and compressed optimizer baselines throughout training, achieving a comparable final cross-entropy loss without convergence degradation. The inset highlights the trajectory from steps 1,150 to 1,500.

Table 12: Zero-shot downstream accuracy (%) of GPT-2 124M checkpoints pretrained under each optimizer, evaluated with the EleutherAI LM-Evaluation-Harness (num_fewshot=0). Bootstrap standard errors shown.

Optimizer	Val Loss	HellaSwag	ARC-C	ARC-E	PIQA	MMLU	Average
Reference FP32 AdamW	4.4581	25.94 $\pm$ 0.44	21.59 $\pm$ 1.20	29.67 $\pm$ 0.94	56.26 $\pm$ 1.16	22.93 $\pm$ 0.35	31.28
FlashAdamW Ortiz et al. (2026)	4.4603	25.79 $\pm$ 0.44	22.18 $\pm$ 1.21	29.00 $\pm$ 0.93	55.50 $\pm$ 1.16	22.94 $\pm$ 0.35	31.08
InfoAdamW (Ours)	4.4587	26.14$\pm$0.44	21.25 $\pm$ 1.20	29.63 $\pm$ 0.94	56.09 $\pm$ 1.16	22.94$\pm$0.35	31.21
Random chance	—	25.00	25.00	25.00	50.00	25.00	30.00

B.6 ZERO-SHOT DOWNSTREAM EVALUATION (FULL RESULTS)

Table 12 reports the full per-benchmark zero-shot results summarized in §4.4, evaluated with the EleutherAI LM-Evaluation-Harness (num_fewshot=0).

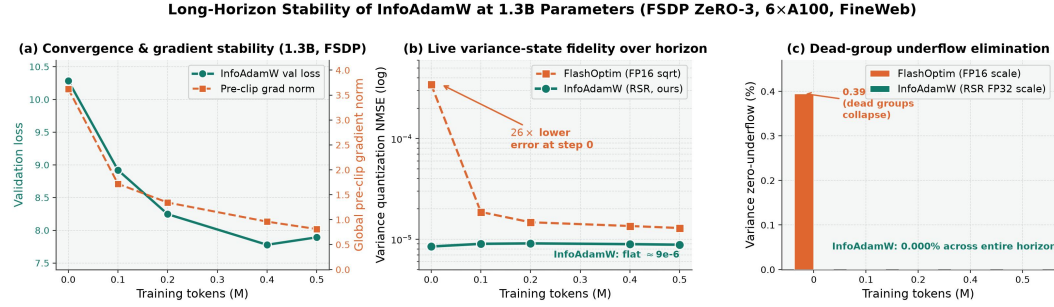


Figure 6: Long-horizon stability of InfoAdamW at 1.3B parameters (FSDP ZeRO-3, 6×A100, FineWeb). (a) Validation loss and global pre-clip gradient norm decrease smoothly over the training horizon with no divergence. (b) Live variance-state quantization NMSE: InfoAdamW (RSR, FP32 scale) stays flat at $\approx 9 \times 10^{-6}$ while FlashOptim’s FP16-scaled square root incurs up to $26\times$ higher error. (c) Variance zero-underflow rate: FlashOptim collapses 0.395% of live variance groups at initialization, whereas RSR maintains 0.000% underflow across the entire horizon.

Table 13: Head-to-head comparison against concurrent low-precision optimizers COAT and ECO on GPT-2 124M (FineWeb, 1,500 steps, single NVIDIA A100). Official implementations under matched hyperparameters.

Optimizer Variant	Val Loss	Weight Mem (MB)	Optim State (MB)	Peak VRAM (MB)	Opt Step (ms)	Full Step (ms)
Reference FP32 AdamW	5.4874	498	996	18,835	13.6	126.7
FlashAdamW Ortiz et al. (2026)	5.4950	249	389	17,112	10.5	109.0
COAT-AdamW (FP8) Xi et al. (2024a)	5.4841	498	272	18,112	11.6	128.7
ECO-AdamW Nikdan et al. (2026)	5.4811	498	502	18,355	137.2	247.5
InfoAdamW (Ours)	5.5023	249	537	17,256	11.9	109.9

Table 14: Hardware performance profiling across kernel fusion granularities (micro-batch size 64, NVIDIA A100 GPU).

Kernel Fusion Granularity	Step Time (ms)	Speedup	Transient VRAM (MiB)	Effective HBM Bandwidth
1. Unfused PyTorch InfoAdamW	235.72	1.0×	1823.5	7.7 GB/s (0.5%)
2. Semi-Fused Triton InfoAdamW	156.22	1.5×	1382.1	11.7 GB/s (0.7%)
3. Fully-Fused InfoAdamW (Ours)	8.16	28.9×	0.0	223.0 GB/s (14.3%)
4. Reference CUDA AdamW (fused)	2.75	85.7×	0.0	1269.2 GB/s (81.6%)
5. FlashAdamW (fused Triton) Ortiz et al. (2026)	9.96	23.7×	0.0	100.0 GB/s (6.4%)

B.7 LONG-HORIZON STABILITY AT 1.3B PARAMETERS (VISUALIZATION)

Figure 6 visualizes the live 1.3B FSDP stability diagnostics summarized in §4.5 (Table 5).

B.8 CONCURRENT LOW-PRECISION OPTIMIZER COMPARISON (COAT & ECO)

Table 13 reports the full head-to-head comparison summarized in §4.4. Both COAT and ECO retain uncompressed 32-bit master weights (498 MB), so despite COAT’s aggressively compressed FP8 optimizer state (272 MB) InfoOptim still attains the lowest peak VRAM by additionally compressing master weights via S2-Split; ECO’s unfused error-feedback write-back inflates its optimizer step by $11.5\times$ relative to InfoOptim’s fused kernel.

B.9 STATE-OF-THE-ART LOW-MEMORY OPTIMIZER COMPARISON (VISUALIZATION)

Figure 7 visualizes the head-to-head comparison of §4.3 (Table 4).

B.10 FULL HARDWARE FUSION PROFILING

Table 14 and Figure 8 report the complete per-granularity breakdown of optimizer step latency, transient VRAM, and effective HBM bandwidth summarized in §4.8.

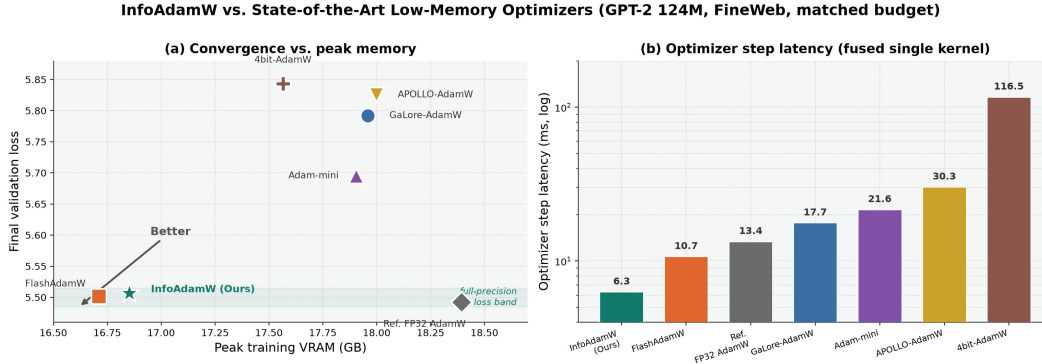


Figure 7: InfoAdamW versus state-of-the-art low-memory optimizers on GPT-2 124M pretraining (FineWeb, matched budget, single A100). (a) Final validation loss against peak training VRAM: InfoAdamW (star) uniquely lands in the full-precision loss band at the lowest memory, while subspace (GaLore, APOLLO), block-sharing (Adam-mini), and 4-bit baselines retain FP32 master weights and converge to substantially higher loss. (b) Optimizer step latency on a log scale: single-kernel fusion makes InfoAdamW the fastest optimizer (6.3 ms), whereas projection and 4-bit dequantization overheads inflate baseline step time up to 116.5 ms.

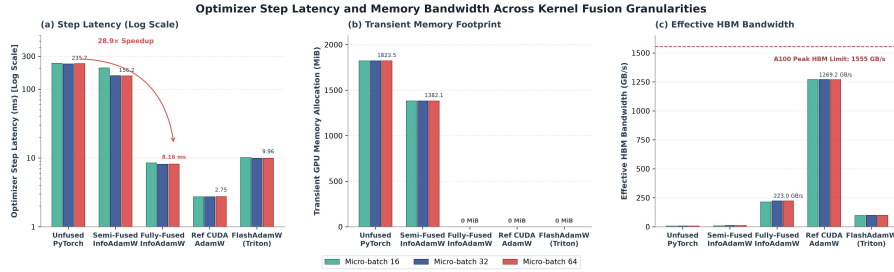


Figure 8: Performance comparison of kernel fusion granularities across micro-batch sizes 16, 32, and 64 for GPT-2 124M pretraining on an NVIDIA A100 GPU. (a) Optimizer step latency on a logarithmic scale, showing a $28.9\times$ speedup for Fully-Fused InfoAdamW over the unfused PyTorch baseline (8.16 ms vs. 235.7 ms at micro-batch 64), achieving throughput comparable to FlashAdamW. (b) Peak transient GPU memory allocation during the optimizer update, demonstrating that full Triton kernel fusion completely eliminates intermediate HBM scratch memory overhead (0 MiB). (c) Effective HBM bandwidth utilization, highlighting that full kernel fusion increases memory throughput to 223.0 GB/s by keeping intermediate state transformations in GPU registers.

B.11 TRITON COMPILER CONFIGURATION & KERNEL PARAMETERS

Fused InfoOptim kernels are compiled using Triton 3.0 with LLVM GPU code generation. Key compilation parameters:

- **Block Tile Size (BLOCK_SIZE):** 1024 elements per CUDA thread block.
- **Warp Count (num_warps):** 8 warps per block (256 threads per block, $G = 32$ elements per warp).
- **Register Limits:** Max 64 registers per thread, avoiding register spilling to local memory.
- **Shared Memory Footprint:** 0 Bytes dynamic shared memory (all reductions executed via intra-warp `_shfl_xor_sync` shuffle instructions).